ACI · AIMLCZG557

Evolutionary Algorithms: GA & ACO

Advanced Computing for AI — Lesson 4

Exam: 28 Jun 2026 · ACO pheromone formula + GA generation trace are key exam items

How to use this companion. Blue = the idea. Amber = everyday analogy. Green = fully-solved example. Red = common mistake. Purple = one line to remember.

1 Genetic Algorithms — The Big Picture

A Genetic Algorithm (GA) mimics natural evolution. You maintain a **population** of candidate solutions (called **chromosomes**). Each chromosome is scored by a **fitness function**. Better-scoring individuals are more likely to be selected as parents. Parents combine via **crossover** to produce offspring. Random **mutation** maintains diversity. Over many generations, the population evolves toward better solutions.

★ Everyday picture

Think of dog breeding. You pick the fastest dogs (selection), mate them (crossover), occasionally a puppy gets an unexpected trait (mutation), and after many generations you have greyhounds. A GA does the same, but the "dogs" are encoded solutions to a problem.

GA Pseudocode (exam-ready):

```
function GA(population, fitness):
  evaluate all individuals
  while not termination_condition:
    parents = SELECT(population, fitness)
    children = CROSSOVER(parents)
    children = MUTATE(children)
    evaluate children
    population = REPLACE(population, children)
  return best_individual_found
```

Where this shows up in ML. GAs are used in Neural Architecture Search (NAS), hyperparameter optimisation, and evolutionary strategies for RL. Google's AutoML-Zero used GAs to discover entire ML algorithms from scratch.

✔ Key takeaway

GA = population + selection + crossover + mutation + replacement. Each iteration is one generation. The fitness function is the only problem-specific part.

2 Selection Methods

Tournament selection. Pick k individuals at random from the population. The one with the highest fitness wins and becomes a parent. Common: $k = 2$ (binary tournament). Higher k = more selection pressure (less diversity).

Roulette wheel (fitness-proportionate). Each individual gets a slice of a wheel proportional to its fitness. Selection probability of individual i:

$$P(i) = \frac{f_i}{\sum_{j=1}^{N} f_j}$$

Problem: if one individual dominates, it gets nearly all the wheel (premature convergence).

Rank-based. Sort by fitness; assign probability based on rank, not raw fitness. Prevents one dominant individual from taking over.

✎ **Worked example: Roulette wheel probabilities**

Population of 4: $f_1 = 10$, $f_2 = 5$, $f_3 = 3$, $f_4 = 2$. Total $= 20$.

Step 1. $P(1) = 10/20 = 0.50$

Step 2. $P(2) = 5/20 = 0.25$

Step 3. $P(3) = 3/20 = 0.15$

Step 4. $P(4) = 2/20 = 0.10$

Individual 1 wins half the time. If it is only slightly better than the others, this may not be ideal — use rank-based selection instead.

✔ **Key takeaway**

Tournament = simple, no sorting, controllable pressure. Roulette = proportional but risky. Rank-based = robust to fitness scaling.

3 Crossover Operators

Crossover combines two parent chromosomes to create offspring.

Single-point crossover. Pick a random cut point. Child 1 = left part of Parent A + right part of Parent B. Child 2 = left part of Parent B + right part of Parent A.

Two-point crossover. Pick two cut points. Swap the middle segment.

Uniform crossover. For each gene position, flip a coin. Child inherits from Parent A with probability 0.5, else Parent B.

✎ **Worked example: Single-point crossover, step by step**

Parent A: `1 1 0 0 1 0 1` Parent B: `0 0 1 1 0 1 0` Cut point: position 3.

Step 1. Child 1 = first 3 genes from A + last 4 from B: `1 1 0 | 1 0 1 0`

Step 2. Child 2 = first 3 genes from B + last 4 from A: `0 0 1 | 0 1 0 1`

Both children inherit traits from both parents. The crossover point is chosen uniformly at random each generation.

4 Mutation

Mutation applies a small random change to an offspring's chromosome. For binary chromosomes, flip each bit with probability p_m (typically $1/L$ where L is chromosome length).

☞ The intuition

Mutation prevents the population from getting stuck. Without mutation, once all individuals share the same allele at a position, crossover can never reintroduce diversity there. Mutation is the only source of truly new genetic material.

✗ Watch out

Mutation rate p_m is a key hyperparameter. Too high: random walk (no learning). Too low: premature convergence. Typical range: 0.001 to 0.01 per bit.

5 Generational vs Steady-State GA

This is a **confirmed exam error** — make sure you know the difference.

Generational GA. The entire population is replaced at each generation. All N parents produce N children; the new population consists entirely of children. Generation boundary is clear.

Steady-state GA. Only a *few* individuals are replaced at a time (often just 1 or 2). The rest of the population survives unchanged. The population evolves incrementally. This maintains more diversity and is more suitable for online/continuous scenarios.

✗ Watch out

The most common exam mistake: saying steady-state GA "replaces the entire population every generation". That is WRONG. Steady-state replaces only a few individuals. Only generational GA replaces the whole population. This distinction appeared in your lesson as a **confirmed error to fix**.

✔ Key takeaway

Generational = replace ALL N individuals each generation. Steady-state = replace only 1–2 individuals at a time. The exam will test this.

6 Crowding & Diversity Maintenance

When many individuals converge on the same region, the population loses diversity (premature convergence). Crowding methods maintain diversity by preventing similar individuals from dominating.

Fitness sharing. Reduce the fitness of an individual based on how many similar individuals are nearby. If many individuals cluster around a peak, each gets a smaller share of that peak's fitness. Formula:

$$f_i' = \frac{f_i}{\sum_{j=1}^{N} \mathrm{sh}(d_{ij})}$$

where $\mathrm{sh}(d) = 1 - (d/\sigma_{\text{share}})^{\alpha}$ if $d < \sigma_{\text{share}}$, else 0. d_{ij} is the distance between individuals i and j.

Deterministic crowding. Each child replaces the most similar parent, keeping the population diverse without explicit distance calculations.

✗ Watch out

A diversity-maximising replacement strategy is NOT the same as generational replacement. Crowding methods (fitness sharing, crowding distance, deterministic crowding) specifically exist to *counteract* the tendency of selection to collapse diversity.

Where this shows up in ML. Quality-Diversity (QD) algorithms in RL (like MAP-Elites) use fitness sharing ideas to maintain a diverse archive of behaviours, not just a single best policy.

7 Ant Colony Optimisation (ACO)

ACO mimics how real ants find the shortest path to food. Ants deposit **pheromone** on paths they travel. Shorter paths accumulate more pheromone per unit time. Other ants preferentially follow high-pheromone paths. Over time, the shortest path gets reinforced.

☞ The intuition

Imagine a colony of ants exploring between nest and food. A short path is completed faster, so ants on it deposit pheromone more frequently. More pheromone attracts more ants, which deposit more pheromone. It is a positive feedback loop that amplifies the best solution.

7.1 Edge Probability

An ant at node i chooses the next node j with probability:

$$p_{ij} = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{k \in \text{allowed}} \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}}$$

- τ_{ij} — pheromone level on edge (i, j). Higher = more attractive (learned).
- $\eta_{ij} = 1/d_{ij}$ — heuristic (inverse distance). Higher = shorter edge (domain knowledge).
- α — pheromone importance. $\alpha = 0$: ignore pheromone (pure greedy heuristic).
- β — heuristic importance. $\beta = 0$: ignore heuristic (pure pheromone following).

7.2 Pheromone Update

After all ants complete their tours, pheromones are updated:

$$\tau_{ij} \leftarrow (1 - \rho)\, \tau_{ij} + \Delta\tau_{ij}$$

- $(1 - \rho)\, \tau_{ij}$ — **evaporation**: reduces all pheromones by factor $(1 - \rho)$ to allow forgetting old paths. $\rho \in (0, 1)$ is the evaporation rate.
- $\Delta\tau_{ij} = \sum_{k=1}^{m} \Delta\tau_{ij}^{k}$ — total new pheromone deposited by all ants on edge (i, j) this iteration.
- $\Delta\tau_{ij}^{k} = Q/L_k$ if ant k used edge (i, j); else 0. Here Q is a constant and L_k is the tour length.

✎ Worked example: ACO pheromone update, one iteration

Two ants, one edge (A, B). $\tau_{AB} = 0.8$, $\rho = 0.3$, $Q = 1$. Ant 1 tour length $L_1 = 10$ (used edge AB). Ant 2 tour length $L_2 = 5$ (did NOT use AB).

Step 1. Evaporate: $(1 - 0.3) \times 0.8 = 0.7 \times 0.8 = 0.56$.

Step 2. Deposit from ant 1: $\Delta\tau_{AB}^{1} = 1/10 = 0.10$.

Step 3. Deposit from ant 2: $\Delta\tau_{AB}^{2} = 0$ (ant 2 didn't use AB).

Step 4. New pheromone: $\tau_{AB} = 0.56 + 0.10 = \mathbf{0.66}$.

The edge pheromone dropped because ant 1 had a long tour. If ant 1 had $L_1 = 2$, deposit

would be 0.50 and τ_{AB} would be $0.56 + 0.50 = 1.06$ — rewarding the shorter tour.

✗ Watch out

Evaporation is applied *before* adding new pheromone. Without evaporation, pheromone accumulates forever and the system converges prematurely to whatever was good early on. Evaporation is the forgetting mechanism.

Where this shows up in ML. ACO has been applied to feature selection, neural architecture search, and combinatorial optimisation (routing, scheduling). Its key insight — positive feedback reinforcing good solutions — is related to bandit algorithms and Thompson sampling.

✔ Key takeaway

ACO: pheromone × heuristic drives edge probability. Update: evaporate first, then deposit inversely proportional to tour length. Short tours = more pheromone = future ants follow.